ITA03 – MOBILE COMPUTING
ASSIGNMENT 4 – CO4 (PART I AND PART II)

Student details

Item	Details
Name	M. Manoj Kumar
Register number	192425114
Course	ITA03 – Mobile Computing
Assignment	Assignment 4 – CO4

How to read this report: Part I is a design and reasoning exercise. Part II needs measured evidence. Items marked MEASURED were produced by the programs listed in Appendix A, run in a Linux sandbox with random seed 192425114 (they are not Android or iOS device results, and the report says so each time). Items marked [DEVICE TEST – record result] must be run on a real phone or emulator and pasted in before submission. No device result is claimed in this report.

PART I – MOBILE APPLICATION DEVELOPMENT ENVIRONMENT

TASK 1 – ARCHITECTURE REVIEW

1.1 Diagnosis of the poor architecture description

The junior engineer's note lists six labels (hardware, kernel, runtime, permissions, battery saving, framework). It cannot be implemented or tested for the following reasons.

#	Deficiency	Why it is insufficient
1	No hardware abstraction or drivers	Does not say how display, storage, sensors, radio and camera are reached, so the driver/HAL boundary is unknown.
2	Kernel role unstated	The kernel's jobs (scheduling, virtual memory, interrupts, device access, security enforcement) are not named.
3	No native libraries or system services	Graphics, media, database, TLS and daemon services that apps rely on are missing.
4	Runtime undefined	Not stated whether code is bytecode or machine code, who loads it, or whether it is interpreted, JIT- or AOT-compiled.
5	Permission enforcement unspecified	No statement of who declares, grants, revokes or checks a permission, or at what boundary.
6	Power saving has no trigger	"Saves battery" names no idle detection, frequency scaling, suspend, wakelock or wake-up rule.
7	No application-to-OS call path	No IPC mechanism, so a request cannot be traced from app to hardware.
8	No process or isolation model	Per-app identity, sandbox, process creation and lifecycle are omitted.
9	No dependency direction	What each layer offers upward and needs from below is not given.
10	Claims cannot be tested	No enforcement point, policy, metric or observable state is named for the security and battery claims.

1.2 Repaired architecture specification (Android)

Android is chosen because its layers are explicit and documented, and because Part II uses Android tools (adb, dumpsys, Perfetto).

Layer (bottom to top)	Responsibility	Interface offered upward	Depends on
Hardware	SoC CPU/GPU, RAM, flash storage, display, sensors, camera, modem and radios.	Registers, interrupts, DMA and bus access.	Physical components and firmware.

Layer (bottom to top)	Responsibility	Interface offered upward	Depends on
Linux kernel + drivers	Scheduling, virtual memory, process and thread management, networking stack, Binder driver, power interfaces (cpufreq, suspend), SELinux enforcement.	System calls, /dev nodes, /proc and /sys.	Hardware and its drivers.
Hardware Abstraction Layer (HAL)	Vendor implementations behind standard interfaces for camera, audio, sensors, Bluetooth, power.	Versioned HAL interfaces (HIDL/AIDL) to system services.	Kernel drivers and firmware.
Native libraries and system daemons	libc, SQLite, OpenSSL-based TLS, media and graphics libraries, SurfaceFlinger, init and servicemanager.	C/C++ APIs and service endpoints.	Kernel and HAL.
Android Runtime (ART)	Loads and verifies DEX, runs code by interpretation, JIT and profile-guided AOT, manages managed heap and garbage collection, provides JNI.	Class loading, execution and memory management for managed code.	Kernel process/memory services and native libraries.
Application framework	system_server services: ActivityManager, WindowManager, PackageManager, PowerManager, NotificationManager, LocationManager.	Java/Kotlin SDK APIs, reached through Binder.	ART, native libraries, HAL.
Applications	Activities, services, receivers and providers running under a per-app identity and lifecycle rules.	User interface and intents/providers to other authorised apps.	Framework APIs.

Explicit locations of the key mechanisms:

- Kernel: Linux kernel layer (process and memory management, Binder driver, drivers, SELinux).
- Runtime: ART, above the kernel and native libraries and below the framework.
- Security enforcement: per-app Linux UID sandbox and SELinux policy (kernel), permission checks in framework services, APK signature verification in PackageManager, key storage in Android Keystore.
- Power management: kernel cpufreq and suspend, Power HAL, PowerManager with Doze and App Standby, and each app's own wakelock and job-scheduling behaviour.
- Application-to-OS path: app calls a framework API, which makes a Binder call to a system_server service; the service calls a native daemon, HAL or kernel driver; the driver talks to hardware.

1.3 Design defence (under 100 words)

The repaired specification is testable because every layer has one bounded responsibility, an upward interface and a named lower dependency. It places the Linux kernel, ART, SELinux and UID sandbox, framework permission checks and power controls at explicit points, and traces an application request through framework API, Binder IPC, system service, HAL and driver. An engineer can therefore locate the component responsible for a fault, write a test for it and judge whether security or power claims hold, unlike a vague list of labels.

TASK 2 – ANDROID vs iOS vs WINDOWS

2.1 Requirement model

Element	Definition for secure background messaging
Input	Each platform's architecture, kernel, runtime, lifecycle, IPC, sandbox and background-execution rules; an incoming encrypted message.
Output	A comparison and a decision on how each platform should receive and show messages reliably.
Objective	Deliver messages securely while the screen is off, with acceptable delay and battery cost.
Constraints	Background limits, network availability, notification permission, end-to-end encryption, process termination, OS version.
Decision	Use each platform's push channel and lifecycle rules; do not copy one permanent background loop to all platforms.
Information required	Push service, process model, IPC path, sandbox, runtime, notification service, background-task rules, secure storage.

2.2 Hypothesis

Delivery reliability will depend more on each OS's background-execution and process-lifecycle policy than on its kernel family, because every platform may suspend or kill an app and relies on a privileged push service to bring messages to it.

2.3 Architectural comparison matrix

Dimension	Android	iOS	Windows 8 / WinRT	Messaging consequence
Kernel	Linux, monolithic with loadable modules.	Darwin XNU, hybrid Mach + BSD.	Windows NT, hybrid with executive services.	All give process, memory and device services; differences are in user-mode policy.
Push service	Firebase Cloud Messaging via Google Play services.	Apple Push Notification service (APNs).	Windows Push Notification Services (WNS).	Each is a privileged system channel; the app should not keep its own socket open.
Runtime	ART running DEX (interpret, JIT, AOT).	Native Mach-O code with Objective-C/Swift runtime and ARC.	WinRT projections to native C++/managed C#.	Affects start-up time and memory when the app is woken.
Framework	Activity, Service, BroadcastReceiver, WorkManager.	UIKit, UserNotifications, BGTaskScheduler.	WinRT app model, background tasks.	Message handlers must be lifecycle-aware.
Isolation	UID sandbox, SELinux, permissions.	App sandbox, code signing, entitlements.	AppContainer, capabilities.	Message data must stay in app-private storage.
IPC	Binder; intents; content providers.	Mach ports and XPC.	Windows IPC and app services; brokered calls.	Push reaches the app only through OS-defined channels.
Background rules	Doze, App Standby buckets, background-start limits.	Suspended soon after leaving foreground; silent push is discretionary.	Apps are suspended; background tasks run only with triggers and quotas.	Visible notification is the dependable path; background code is best effort.

2.4 Incoming message desk-check

Step	Android	iOS
1. Network event	Sender posts the message to FCM; FCM sends it over a persistent system connection.	Sender posts to APNs; APNs sends it over a persistent system connection.
2. Kernel / radio	Modem and network driver pass packets up the kernel network stack to Google Play services.	Baseband and Darwin network stack pass packets to the system push daemon.
3. System service	Google Play services hands the message to the target app, or posts a notification, following priority and standby rules.	The push daemon applies notification permission and Focus/power policy.
4. App wake	For a high-priority message the system may start the app process for a short window.	The alert is shown; a silent push may or may not launch the app, and is not guaranteed.
5. Runtime	Zygote forks the process and ART loads the class that handles the message.	Native code runs the permitted notification-service or background handler.
6. Application	App decrypts and stores the message in private storage, then updates the notification.	App decrypts and stores the payload, then updates the notification.

2.5 Changed condition: application terminated

Prediction: on Android an eligible high-priority push can recreate the process, unless the user force-stopped the app or the vendor's battery manager blocks it. On iOS a visible notification is shown without launching the app, and silent background launch is discretionary. On Windows, a WNS toast can be shown, and background code runs only if a registered background task trigger fires. The deciding elements are the push service, the lifecycle manager and the background-execution policy.

This prediction must be checked on a device. No test logs are supplied, so no observed result is claimed.

TASK 3 – RUNTIME ENVIRONMENT ANALYSIS

3.1 Execution model

Platform	Delivered form	Loader	When machine instructions exist
Android / Dalvik (legacy)	classes.dex in the APK	PackageManager and Dalvik VM in a Zygote-forked process	Bytecode is interpreted and hot code is JIT-compiled at run time.
Android / ART (current)	DEX plus compiled artifacts and cloud/local profiles	Zygote-forked process with ART class loader	Mix of interpretation, JIT and profile-guided AOT; depends on compilation state.
iOS / Objective-C runtime	Signed Mach-O binary and frameworks	launchd and the dyld dynamic linker	Machine code is produced at build time; run time does dynamic linking and message dispatch.
Windows / WinRT	Native PE binaries or managed assemblies in an AppX package	Windows app activation and the language runtime loader	Native code is loaded as machine code; managed code is JIT- or NGEN-compiled by the CLR.

3.2 Android application launch sequence

Order	Action	Component	Prerequisite
1	User taps the icon; the launcher sends a start-activity request.	Launcher, ActivityManager	Package installed and valid.
2	ActivityManager checks whether the app has a process.	ActivityManager	system_server running.
3	If none, it asks Zygote to fork a process with the app's UID and SELinux context.	Zygote, kernel	Zygote running; kernel can create processes.
4	ART starts in the new process, loads classes and verifies DEX as needed.	ART	Process and runtime libraries present.
5	The Application object is created and bound to the framework.	Framework	ART initialised.
6	Activity callbacks run: onCreate, onStart, onResume.	ActivityManager, app main thread	Activity class loaded.
7	Views are measured, laid out and drawn, and the frame buffer is submitted.	ViewRootImpl, RenderThread, SurfaceFlinger	Window and graphics services available.
8	SurfaceFlinger composes the frame and the display shows it.	SurfaceFlinger, display hardware	Buffer ready; display driver active.

Each step needs the one before it. The kernel must create and schedule the process before ART can start; ART must load classes before callbacks run; window and graphics services must exist before a frame can be shown.

3.3 Runtime strategy comparison

Aspect	Dalvik (legacy Android)	Objective-C / Swift on iOS	Windows runtime
Execution	Interpreter plus trace JIT on DEX.	Native code; Objective-C message dispatch through objc_msgSend.	Native or managed, chosen by language.
Compilation	JIT at run time; no whole-app AOT (ART adds it).	Ahead-of-time by the compiler and linker.	Native AOT; managed code compiled by the CLR.
Memory	Managed heap, tracing GC.	ARC (reference counting), no tracing GC.	Manual or RAII in native; GC in managed.
Responsiveness	GC pauses and JIT warm-up can drop frames.	No GC pauses; retain/release cost, reference cycles leak.	Depends on language and GC settings.
Battery	JIT and GC use CPU.	Efficient if ownership is correct.	Depends on allocation and scheduling.
Portability	One DEX runs on several CPU architectures.	Binary built for Apple architectures only.	Binaries built per architecture and API set.

3.4 Suitability for a low-memory device

A small ahead-of-time compiled native program with predictable object ownership is the strongest candidate for very little memory, because it needs no compiler in memory and no tracing collector with headroom. A managed heap is simpler and safer but needs spare heap for the collector. ARC avoids collector pauses but needs care with cycles. No runtime is smallest in every case; the choice must be confirmed by measuring PSS (proportional set size) on the target.

3.5 Cold-start and warm-start desk-check

Stage	Cold start	Warm start
Process creation	Required.	Skipped; process still alive.
Runtime initialisation	Required.	Already done.
Class loading	Classes loaded on first use.	Most classes already loaded.
Application object	Created.	Reused.
Activity	Created, full callback sequence.	Resumed, or recreated if the system destroyed it.
First frame	Whole view tree built and drawn.	Often just redrawn from cached state.

3.6 Dependency analysis

The kernel process and memory managers must exist before a process can; the runtime must start before managed classes run; framework services must exist before lifecycle callbacks can ask for windows, permissions or notifications; and the compositor must be ready before a frame can reach the screen. This order prevents a layer from calling an interface that has not been created yet.

TASK 4 – SECURITY FORENSICS: SECURENOTES

4.1 Defects identified

Original decision	Mechanism bypassed	Risk
Request every permission at install for future use	Least privilege and runtime consent	Larger attack surface; a compromised app can use access the user never meant to give.
Store login token in a file readable by other apps	App sandbox and secure storage	Another app can copy the token and impersonate the user.
Accept updates from any source	Code signing and trusted update chain	A forged package can replace the app’s code.
Use plain HTTP to the server	TLS confidentiality, integrity and server authentication	Anyone on the network can read or change credentials and notes.

4.2 Attack scenario

A user at a café joins an attacker’s Wi-Fi hotspot while a malicious app is installed on the phone.

1. SecureNotes logs the user in and writes the token to a world-readable file.
2. The malicious app reads the file. Confidentiality is lost at this step (first failure).
3. The attacker replays the token to the server over the open network and reads or edits notes.
4. SecureNotes checks for an update over plain HTTP.
5. The attacker’s hotspot answers with a modified package, and because no signature check is made the app accepts it. Integrity is lost at this step.
6. The malicious update runs with SecureNotes’ permissions, including every permission requested at install, and keeps stealing data.

4.3 Minimal repairs for Android

Defect	Repair	Android mechanism and layer
Excess permissions	Declare only what features need; ask for dangerous permissions at the moment of use.	Manifest and runtime permission dialogs; checks in framework services.

Defect	Repair	Android mechanism and layer
Shared token file	Keep the token in app-private storage, encrypted with a key held in Keystore.	UID sandbox and SELinux (kernel); Keystore (system service backed by TEE/StrongBox).
Unverified updates	Update only through a trusted installer and require the same signing certificate.	PackageManager signature check, Play app signing.
Plain HTTP	HTTPS with certificate validation; cleartext disabled.	Network Security Config and the platform TLS provider.

4.4 Regression check

Legitimate feature: the user taps “Import contact”. The app declares READ_CONTACTS, asks at that moment, reads only the chosen contact and handles a denial without crashing. The token is still in private encrypted storage, updates are still signature-checked, and traffic is still TLS, so the repairs do not break the consent-based import.

4.5 Security verdict

The original design must not be released. The minimum release conditions are least-privilege permissions, private and encrypted token storage, signed updates from a trusted channel, and TLS everywhere. Residual risks are rooted devices, a compromised OS, malicious accessibility services, a stolen server credential and flaws in third-party libraries. They need monitoring, secure development practice and an incident-response plan.

TASK 5 – POWER MANAGEMENT REASONING

5.1 Power-state model

State	Hardware and kernel behaviour	Entry	Exit	Work allowed
ACTIVE	Screen on, CPU scheduled normally, sensors at configured rate, radio available.	User interaction, active workout, foreground tracking.	Screen timeout or lock.	UI, sensor sampling, short processing, network calls.
IDLE	Screen off or dim, CPU frequency lowered, timers and sensors batched, background limited.	No interaction for a set time and no foreground work.	User touch, scheduled job, push, motion.	Short jobs, batched sensor events.
SUSPENDED / DEEP IDLE	Cores in low-power states, most app code paused, only wake sources active.	No wakelock and no pending work; Doze conditions met.	Alarm, push, user, hardware wake source.	System maintenance; app work only in maintenance windows or when allowed.

5.2 Urgent push while suspended

The unsafe assumption is that the fitness app must hold the CPU awake all the time so it never misses a sensor event. That blocks deep idle and drains the battery. Instead, urgent alerts should come through the platform push service, which wakes the device for a short bounded time. Sensor data should be batched in hardware FIFO and timestamped, and any partial wakelock should have a timeout and be released on every success and failure path.

5.3 Battery saver at 15%

Layer	Policy change at 15%
Kernel / hardware	Cap CPU frequency (DVFS), prefer efficiency cores, reduce wakeups, allow deeper idle.
OS services / runtime	Restrict background execution, defer non-urgent jobs, batch alarms and network work.
Application	Lower sensor rate when stationary, upload in batches, reduce animations, keep urgent event handling.
User-visible trade-off	Route and statistics update less often when stationary; emergency and user-requested actions still run immediately.

Battery saver must never silently discard data: readings stay timestamped in a local queue until they can be uploaded.

TASK 6 – RURALCARE FIELD-DATA COLLECTION SYSTEM

6.1 Stage 1 – Problem interpretation

Element	Definition
Input	Three device classes, patient data, worker actions, battery, connectivity, storage and security features.
Output	Platform-specific capture, secure offline storage and reliable synchronisation.
Objective	Finish a ten-hour shift, protect patient data, and sync every record with no loss and no duplicates.
Constraints	Class A Android, Class B iPhone, Class C J2ME or WML feature phone; hours offline; limited battery and storage; devices can be lost.
Decisions	Native or web client per class, local data format, encryption, sync protocol, retry and retention rules.

6.2 Stage 2 – Platform strategy

Class	Strategy	Reason
A – Android	Native Kotlin app: Room/SQLite with SQLCipher or field-level AES-GCM, Keystore key, WorkManager, TLS.	Strong offline store, hardware-backed keys, runtime permissions and managed background work.
B – iPhone	Native Swift app: Core Data/SQLite with Data Protection, Keychain, BGTaskScheduler, URLSession background transfer.	Keychain and file protection secure local data; background work is controlled by iOS.
C – J2ME	Small MIDlet using RMS as a queue; encrypt records only if the handset offers a cryptography API; sync in small batches.	Tiny storage and limited security; keep as little data as possible.
C – WML only	Server-rendered WML forms over HTTPS if available; patient records stay on the server.	No trusted local storage can be assumed.

6.3 Stage 3 – Constraint table

Class	Runtime	Security	Power control	Offline storage	Connectivity
Android	ART, Kotlin/Java	UID sandbox, SELinux, runtime permissions, Keystore, file-based encryption	Doze, App Standby, WorkManager, foreground service when justified	Encrypted Room/SQLite	Wi-Fi, cellular, offline queue with retry
iOS	Swift/ObjC, ARC	Sandbox, signing, entitlements, Keychain, Data Protection	Background App Refresh, BGTaskScheduler, system suspension	Protected Core Data/SQLite	Wi-Fi/cellular; background transfer subject to iOS policy
J2ME	Java ME / MIDP	Device-specific MIDlet permissions; app-level protection	Device pause; low-frequency polling	RMS record store, small	GPRS/2G/3G, intermittent
WML	Browser	Server-side authentication; HTTPS if supported	Browser and device managed	None for sensitive data	Low-bandwidth data connection

6.4 Information kept for safe synchronisation

Field	Purpose
Record UUID	Lets the server recognise a retried record and avoid duplicates.
Worker ID and device ID	Origin and accountability.
Patient reference	Links to the patient without copying identity data.
Device timestamp, time zone, monotonic sequence number	Event order even if the clock changes.
Schema version and record version	Compatibility and conflict detection.
Payload hash (SHA-256)	Detects corruption in transfer.

Field	Purpose
Sync status (PENDING, SENDING, ACKED, FAILED)	Shows what still has to be sent.
Retry count and next-attempt time	Controls back-off.
Server acknowledgement ID	Proof the server committed that record.
Key version	Decryption and key rotation.

6.5 Stage 4 – Complete solution specification

Layered architecture

```
FIELD DEVICE
  Presentation : patient form, validation, worker login
  Domain       : record creation (UUID), sync state machine
  Data         : encrypted local queue + transaction log
  Security     : device-bound key, AES-GCM, access control
  OS services  : WorkManager / BGTask / J2ME timer, network + battery state
          | HTTPS (TLS)
SERVER  : API gateway -> auth -> validation -> idempotency check
          -> transactional DB -> acknowledgement -> audit log
```

Capture and offline storage

1. Authenticate the worker and load only the patient data needed for the visit.
2. Validate the form on the device.
3. Create a UUID and timestamps.
4. Encrypt the record, then write it to the local queue; the key stays in secure storage.
5. Save the record and its PENDING status in a single database transaction.
6. Show a pending-sync indicator; never show patient details on the lock screen.

Security policy

- Android: private database, AES-GCM data key wrapped by Keystore, device-credential gate when policy requires.
- iOS: secrets in Keychain, files under a suitable Data Protection class, TLS with server authentication.
- J2ME: store minimal data; encrypt only when a key store and crypto library exist, otherwise restrict offline content.
- WML: no sensitive local storage; short authenticated HTTPS sessions.
- Lost device: revoke the device credential, disable API access, remote-wipe if managed, rotate tokens, audit the last successful sync.

Ten-hour power policy

Condition	Capture policy	Sync policy
Normal battery, moving	Sample at the configured interval, timestamped.	Batch and upload when online.
Stationary	Cut redundant location samples; manual entries unaffected.	Upload in scheduled windows.
Low battery	Longer non-critical interval; manual clinical records always allowed.	Defer bulk sync; urgent records still go.
No connectivity	Keep writing to the encrypted queue.	Exponential back-off; no tight retry loop.
Charging + Wi-Fi	Normal sampling.	Bulk sync and verify acknowledgements.

Synchronisation procedure

1. Check network, token validity, battery and sync eligibility.
2. Read a bounded batch of PENDING records in creation order.
3. Send each with its UUID, schema version and payload hash over TLS.
4. Server authenticates device and worker and validates the payload.

5. Server looks up the UUID; a repeat returns the earlier acknowledgement instead of inserting again.
6. Server commits in one transaction and returns an acknowledgement ID.
7. Client marks only acknowledged records ACKED.
8. On interruption, records stay PENDING and are retried with back-off.
9. After a restart, resume from the queue; sending a request is never treated as success.
10. Keep audit metadata for the retention period.

Failure handling

Failure	Handling
App killed	Every record is saved before the screen shows “saved”; the queue resumes on restart.
Network lost	Encrypted records stay queued; bounded exponential back-off.
Duplicate request	Server idempotency on the UUID prevents a second insert.
Corrupt payload	Hash check rejects it; local copy is kept and flagged for retry.
Phone lost	Revoke token, remote wipe if enrolled, rotate credentials.
Feature phone without secure storage	Minimum data, short sessions, restrict offline workflow.
Storage full	Stop new captures before overwriting any PENDING record; warn the worker.

6.6 Stage 5 – Desk-check: Class A, 6 hours offline, battery 20%, then device lost

Step	State / decision	Exposure and recovery
1	Worker signs in; app unlocks its Keystore key and opens the encrypted queue.	No plaintext patient data is visible to other apps.
2	Six hours offline; records are encrypted and PENDING.	Data exists only on the device; server has no copy.
3	Battery reaches 20%; non-critical sampling is lengthened, manual records are kept.	Nothing is discarded.
4	Device is lost before any sync.	Records without acknowledgement cannot be recovered from the server.
5	Administrator revokes the device token and sends a remote lock/wipe if enrolled.	Server access is blocked; wipe works only if the phone gets online.
6	If the finder tries to read storage.	Encrypted at rest; plaintext needs the Keystore key, which is bound to the device and screen lock.

Exposure: low for a locked, non-rooted device with a hardware-backed key; higher if it was unlocked, rooted or the credentials were already stolen. Loss: all records not yet acknowledged may be lost permanently. Recoverable: everything already acknowledged by the server.

6.7 Stage 6 – Adversarial tests

Adverse condition	Prediction	Refinement
Stolen unlocked phone	Attacker can use the open app.	Re-authenticate for sensitive actions, short auto-lock, hide notification content, revoke remotely.
Connection drops after server commit but before client acknowledgement	Client retries; a duplicate is possible without idempotency.	Server returns the earlier acknowledgement for the same UUID.
Class C handset with no secure storage	Persistent patient data would be exposed.	Do not cache full records; server-side workflow; offline requirement cannot be met on this device.
Storage full	New records fail or old ones are overwritten.	Reserve space, monitor queue, never overwrite PENDING records.
Device clock changed	Order and audit time unreliable.	Keep device timestamp plus monotonic sequence number and server receipt time.

6.8 Final evaluation

One native app cannot give all three device classes the same security and offline capability. Android and iPhone can hold an encrypted offline database and sync safely. J2ME can hold a limited queue only if the handset supports protection. WML-only phones cannot be trusted with persistent offline storage, so their data retention must be limited or the workers given a managed device. No-duplicate sync rests on unique record IDs, server-side idempotency, transactional commits and explicit acknowledgements.

PART II – MOBILE SYSTEMS ENGINEERING

Evidence rules used in this part: MEASURED = produced by my programs in the sandbox (Linux, Python 3.12.3, 1 virtual CPU, seed 192425114). This is a desktop environment, so it demonstrates a principle but is not an Android result. [DEVICE TEST] = to be run on an Android phone or emulator and pasted in.

TASK 1 – ANDROID vs iOS RESOURCE-MANAGEMENT MODEL

1.1 Layer-by-layer responsibility

Layer	Android	iOS
Kernel	Linux: scheduler, virtual memory, drivers, networking, Binder driver, SELinux, low-memory killer.	XNU: Mach tasks and threads, BSD layer, virtual memory, networking.
Hardware interface	HAL implementations over kernel drivers.	IOKit and driver families.
Runtime	ART: DEX execution, JIT/AOT, class loading, garbage-collected heap.	Objective-C/Swift runtime with ARC.
Framework	ActivityManager, WindowManager, PackageManager, NotificationManager.	UIKit, UserNotifications, BackgroundTasks.
Process creation	Zygote forks the app process and sets UID and SELinux context.	launchd and the kernel launch the app; dyld links it.
IPC	Binder.	Mach ports and XPC.

1.2 Evidence commands (Android) [DEVICE TEST]

```
adb shell ps -A | grep <package>
adb shell dumpsys activity processes | grep -A3 <package>
adb shell dumpsys meminfo <package>
adb shell cat /proc/<PID>/oom_score_adj
adb shell dumpsys package <package> | grep -E 'userId|versionName'
adb logcat -v time | grep <package>
```

How to read them: ps gives PID, user (the per-app UID) and state; dumpsys activity processes shows the process's importance (foreground, visible, cached); dumpsys meminfo shows PSS by category; oom_score_adj is the value the low-memory killer uses, and a higher value means the process is killed earlier.

1.3 Lifecycle and memory-condition test matrix

Condition	Prediction	Evidence to record	Result
Foreground	Process stays alive; oom_score_adj near 0; high importance.	PID, process state, lifecycle log, PSS.	[DEVICE TEST – record result]
Background 5 minutes	Becomes cached; may be frozen, trimmed or killed depending on memory.	dumpsys activity processes, oom_score_adj (expect a higher value), PID unchanged or not.	[DEVICE TEST – record result]
Low memory	Cached processes are killed first; foreground is protected.	PID before and after, onTrimMemory levels, state-restore logs.	[DEVICE TEST – record result]

1.4 Instrumentation and interpretation

Log onCreate, onStart, onResume, onPause, onStop, onDestroy and onSaveInstanceState with the PID and a timestamp. When the system kills a background process there is no onDestroy call, so absence of that log line is itself evidence of process death. State must therefore be saved in onStop (or with SavedStateHandle) and not in onDestroy.

1.5 Boundary test

Rotate the screen while a form holds unsaved text and a request is running. Expected safe behaviour: the form text is restored from saved state, the request is not duplicated, and the worker holds no reference to the destroyed Activity. [DEVICE TEST – record result]

iOS evidence needs macOS, Xcode and an iPhone or simulator; none was available, so no iOS command output is claimed.

TASK 2 – ARCHITECTURE ASSUMPTIONS vs MEASURED REALITY

2.1 Two implementations

Solution	Design	Purpose
A – single thread	Parse and filter the dataset on the main (UI) thread.	Baseline; shows the risk of blocking the UI.
B – background executor	Same algorithm on a bounded worker pool or coroutine; only the result goes to the UI thread.	Keeps the UI responsive.

2.2 Measurement plan [DEVICE TEST]

- Same dataset and algorithm for A and B; warm up before timing.
- Record wall time, CPU time, thread count, context switches, PSS/heap, allocations and frame timing.
- Use Android Studio Profiler or Perfetto for thread scheduling and CPU placement; use dumpsys gfxinfo <package> framestats for dropped frames.
- Run each case at least 5 times; report median and range.

```
adb shell top -H -p <PID>
adb shell dumpsys meminfo <package>
adb shell dumpsys gfxinfo <package> framestats
```

2.3 Counter-example: more threads can be slower (MEASURED in sandbox)

Task 1: 5,000 tasks, each summing a 100-element list. Times are in milliseconds for the whole batch, median of 5 runs.

Strategy	Median (ms)	Range (ms)
single thread	3.07	2.76 – 4.09
new thread per task	157.14	144.34 – 178.41
pool of 4 threads	47.93	40.22 – 50.75

Creating a thread for every tiny task took about 51 times longer than the single thread, and a pool of 4 threads still took about 16 times longer, because creation, scheduling and hand-off cost more than 100 additions.

Task 2: 400,000 increments of one counter. Single thread without a lock against 4 threads sharing one lock:

Strategy	Median (ms)	Range (ms)
single thread, no lock	9.67	9.6 – 11.19
4 threads, one shared lock	73.37	72.37 – 75.27

The lock forces the four threads to run one at a time and adds hand-off cost, so the shared-lock version was about 7.6 times slower.

Task 3: four CPU-heavy tasks (1.5 million loop steps each). The sandbox has only 1 virtual CPU, so no parallel speed-up is possible:

Strategy	Median (ms)	Range (ms)
sequential (4 tasks)	313.3	295.1 – 423.1
4 threads	334.7	310.8 – 353.1
4 processes	330.5	299.2 – 437.2

This confirms the hypothesis that extra threads or processes need extra cores to help CPU-bound work. Python's global interpreter lock would also stop threads running Python code in parallel, so this exact result does not transfer to Kotlin threads on an 8-core phone, which must be measured separately.

Metric (Android) [DEVICE TEST]	A – single thread	B – executor	Result
Task duration (median)	record	record	record
CPU time	record	record	record
Thread count	record	record	record
PSS / heap	record	record	record
Dropped frames	record	record	record
Energy estimate	record	record	record

2.4 First degrading decision

In the tiny-workload case the first harmful decision is starting a thread for work cheaper than thread creation and hand-off. In the lock case it is putting all shared mutable state behind one hot lock. The remedies are a persistent bounded executor, batching tasks, and less shared state.

2.5 Changed requirement: battery matters for a full working day

Use a small bounded pool, batch the parsing and sensor processing, hand non-urgent work to WorkManager with network and charging constraints, avoid busy loops and thread creation, and keep urgent tracking events on a separate path. The final choice should be justified by the measured CPU time, wakeups, energy estimate and frame timing from the device test.

TASK 3 – RUNTIME BEHAVIOUR UNDER CHANGING CONDITIONS

3.1 Benchmark workloads

Workload	What it measures	Expected runtime sensitivity
CPU-bound loop	Steady-state arithmetic and branches.	JIT/AOT quality for hot code.
Allocation-heavy routine	Allocation rate, heap growth, garbage collection.	GC frequency and pause time.
List rendering	Frame time and scroll smoothness.	Main-thread work, object churn, layout cost.

3.2 ART compilation commands [DEVICE TEST]

```
adb shell cmd package compile -m verify -f <package>
adb shell cmd package compile -m speed-profile -f <package>
adb shell cmd package compile -m speed -f <package>
adb shell dumpsys package <package> | grep -A2 'Dexopt state'
adb logcat -v time | grep -i -E 'art|gc|heap'
adb shell dumpsys meminfo <package>
```

Compiler filter names and availability change with Android versions; record the exact output and the build number.

3.3 Prediction before changing compilation mode

Mode	Prediction for CPU workload	Start-up / storage	Result
verify	Hot code runs more in the interpreter/JIT, so steady-state is slowest.	Least compile work and storage.	[DEVICE TEST]
speed-profile	Hot methods named in the profile are compiled.	Balanced.	[DEVICE TEST]
speed	Most code compiled ahead of time; fastest steady state.	Larger compiled files and longer install/compile time.	[DEVICE TEST]

3.4 GC and heap evidence

Record the heap size before and after the allocation workload, the number of GC events, GC pause times and the frame timing during the run. A heap that keeps growing and does not fall back after GC points to a leak or an unbounded cache. Many short GC cycles that free little point to allocation pressure.

3.5 Production failure injection

- Leak: keep a static reference to an Activity, or grow a cache with no eviction. Detect with LeakCanary, heap dump or rising PSS.
- Main-thread blocking: sleep or parse a large file on the UI thread in a test build. Detect with StrictMode, an ANR trace or janky frames.
- Correction: remove the static reference, bound the cache (for example LruCache), move blocking work to an executor and post only the result to the UI thread.

3.6 Cross-platform runtime comparison

Runtime	Execution and memory	UI thread and native access	Trade-off
Native Android (ART)	DEX with interpret/JIT/AOT, tracing GC.	UI on main thread; JNI to C/C++.	Best platform fit; GC and runtime cost need measuring.
Native iOS (Swift, ARC)	Compiled code; reference counting.	UIKit on main thread; direct framework calls.	No GC pauses; reference cycles leak.
Flutter / Dart	AOT in release builds; Dart GC.	Own rendering engine; platform channels for native APIs.	Same UI on both platforms; engine adds size.
React Native / WebView	JavaScript engine; native modules or JSI; WebView adds a browser.	UI work coordinated with native main thread.	Fast shared development; bridge and JS memory cost.

Provisional recommendation: native apps when secure storage, predictable background behaviour and deep platform access matter most (as in RuralCare); Flutter if one code base is worth the engine footprint; React Native or WebView only after measuring on target devices. This stays provisional until the device benchmarks above are filled in.

TASK 4 – SECURE MOBILE APPLICATION AUDIT

4.1 Audit inputs

Input	Checks
Manifest / Info.plist	Excess permissions, missing runtime consent, debuggable or backup flags.
Data stores	Plaintext secrets, public files, unencrypted databases, weak key handling.
Network endpoints	HTTP use, TLS validation, certificate trust.
Exported components and URL schemes	Unprotected activities, services, receivers and deep links.

4.2 STRIDE threat model

Threat	Asset / vector	Protection	Residual risk
Spoofing	Stolen token or forged client.	Signing, strong authentication, Keystore-bound tokens.	Stolen unlocked session.
Tampering	Edit local DB or API payload.	Encrypted storage, integrity checks, TLS, server validation.	Rooted device.
Repudiation	Worker denies a change.	Server audit log with user, time and record ID.	Stolen credentials.
Information disclosure	Read token or DB; sniff traffic.	Sandbox, encryption, TLS.	OS compromise, screenshots.
Denial of service	Request floods, storage exhaustion.	Rate limits, bounded queues, back-off.	Large outage.
Elevation of privilege	Exported component, broad permission.	exported=false, signature permissions, least privilege, SELinux.	OS vulnerability.

4.3 Asset decision table

Asset	Attack vector	Mechanism	Result	Reason
Auth token	Read app files or logs.	Keystore-encrypted storage; no token logging.	Partially blocked	Protects data at rest; a hijacked running process can still use it.
Local DB	Copy the database file.	Sandbox, encrypted DB, key in secure storage.	Partially blocked	Strong on normal devices; rooted OS remains a risk.
API traffic	Man-in-the-middle.	TLS validation, pinning if justified.	Blocked (valid trust)	TLS protects confidentiality and integrity.
Exported activity	Crafted intent from another app.	exported=false; validate caller and input.	Blocked	Other apps cannot start a non-exported component.

4.4 Predict-before-execute experiment (MEASURED on a sample manifest)

Prediction: if PatientEditActivity is changed from exported=false to exported=true with no protection, the audit will change from “no component findings” to a HIGH finding. I wrote a small Python auditor (Appendix A) and ran it on a sample manifest for a RuralCare-style app, before and after the change.

Before (4 findings):

- [HIGH] Unneeded dangerous permission READ_CONTACTS
- [HIGH] Unneeded dangerous permission READ_SMS
- [HIGH] Cleartext HTTP allowed
- [MEDIUM] allowBackup=true exposes app data to adb backup

After setting exported=true on PatientEditActivity (5 findings):

- [HIGH] Unneeded dangerous permission READ_CONTACTS
- [HIGH] Unneeded dangerous permission READ_SMS
- [HIGH] Cleartext HTTP allowed
- [MEDIUM] allowBackup=true exposes app data to adb backup
- [HIGH] Exported component without launcher role: .PatientEditActivity

The prediction held: the only new finding after the change is the exported component. The other four findings (two unneeded dangerous permissions, cleartext traffic and allowBackup) were deliberately placed in the sample and remain to be fixed. This was a static test of a sample file; on a real device, repeat it with adb and record the output. [DEVICE TEST]

```
adb shell am start -n <package>/.PatientEditActivity
adb logcat -v time | tail -n 30
adb shell dumpsys package <package> | grep -A3 PatientEditActivity
```

Restore exported=false afterwards, or add a signature permission and strict input validation.

4.5 Third-party library leaks a device identifier

Asset: the device identifier and related user and device metadata. Affected boundary: the app's own networking layer. The sandbox and permission system cannot stop a library running inside the app's trusted process from sending out data the app can read. Detection: dependency and SBOM review, static analysis, traffic capture through a proxy on a test device, runtime instrumentation. Remediation: remove or update the library, stop collecting the identifier unless essential, use an egress allow-list, and add an automated network test. TLS alone does not help, because the library may send to its own server over TLS.

4.6 Biometric login and BYOD extension

- Use the biometric prompt to unlock a Keystore or Keychain key; never store biometric data in the app.
- Add to the threat model: spoofing, passcode fallback, enrolment changes and session theft.
- Use key attestation where available to confirm the key is hardware-backed.
- Separate work and personal data with an Android work profile or iOS managed apps.
- Limit remote wipe to the work data and credentials.
- Extend the audit to check profile boundaries, managed configuration, key protection and data-export paths.

TASK 5 – THREE POWER ENGINEERS, ONE DEVICE

5.1 Technique comparison

Technique	Trigger and state	Expected effect	Side effect
DVFS / CPU governor	CPU load, temperature, power policy; keeps utilisation, frequency, voltage.	Lower frequency and voltage at light load cuts CPU energy.	Slower work; thermal limit can cut sustained speed.
Doze / App Standby	Screen off, device still, inactivity; keeps idle state and app bucket.	Defers jobs, alarms and network; fewer wakeups.	Delayed sync and notifications.
Wakelock discipline and batching	App holds a bounded wakelock or schedules batched work; keeps lock time, job constraints, pending data.	Fewer CPU and radio wakeups.	Higher latency; a long wakelock cancels the saving.

5.2 Instrumentation [DEVICE TEST]

```
adb shell dumpsys batterystats --reset
# run one controlled test
adb shell dumpsys batterystats > stats.txt
adb shell dumpsys deviceidle
adb shell dumpsys jobscheduler
adb shell dumpsys power | grep -i wake
```

Use Battery Historian or Android Studio's energy profiler for wakelock time and radio-active time. Record the device model, Android build, battery level, screen state, network type, test length and workload, and repeat each run.

5.3 Conflict test

Conflict 1: an app holds a partial wakelock all the time while the OS tries to enter Doze. Prediction: the CPU cannot reach deep idle, so Doze saves little. Conflict 2: an app sets frequent exact alarms, which keep waking the device and break batching. The first decision causing the conflict is the app's unnecessary continuous wake request or high-frequency alarm. [DEVICE TEST – record batterystats before and after]

5.4 Deployment decision

Environment	Emphasis	Reason
A – always-on location/sensor tracking	Foreground service only when justified, sensor batching, adaptive sampling, tight wakelocks.	Tracking cannot be skipped, so cut fixed wake-up cost with batching and adaptive intervals.
B – media / gaming	DVFS and thermal-aware scheduling; keep rendering smooth.	Sustained CPU/GPU and display load dominate.
C – large managed fleet	WorkManager constraints, managed policies, OS-version handling, fleet telemetry.	One policy must work across versions, battery states and enterprise limits.

A technique at one layer can be defeated by another: Doze can defer background work, yet an app that holds a wakelock or sets frequent alarms keeps the device awake. Power management therefore has to be coordinated across hardware, kernel, OS and application.

TASK 6 – BATTERY-AWARE, SECURE FIELD-TRACKING PROTOTYPE

6.1 Problem model

Element	Definition
Input	Battery level, charging, network type, motion, temperature, queue size and age, emergency flag.
Output	Sampling interval, upload action, queue state, encryption status.
Objective	Maximise useful records delivered per unit of energy while keeping data protected.
Constraints	Full shift, changing connectivity, thermal limits, secure storage, OS background limits, urgent delivery.

Battery percentage alone is not enough: two phones at 20% can need different actions, for example one still on Wi-Fi with a large queue and the other moving on LTE during an emergency.

6.2 State-based policy

Dimension	Values
Battery	HIGH (>50%), NORMAL (16–50%), LOW (≤15%), CRITICAL (≤5%), CHARGING
Network	WIFI, CELLULAR (LTE), OFFLINE
Motion	MOVING, STATIONARY
Other	HOT (thermal), EMERGENCY

Default state: NORMAL battery, OFFLINE network, STATIONARY motion until sensors report. Every record is encrypted before it is queued, and every upload uses authenticated TLS.

6.3 Layered architecture

```
UI (main thread)       : map, battery and sync status
Domain (executor)     : policy engine, sampling controller, sync state machine
Data                  : encrypted local queue + transaction store
Security              : AES-GCM record encryption, Keystore-protected key
OS services           : Fused location / sensors, foreground service (only when
                        user-visible continuous tracking is needed), WorkManager
Network               : TLS API with idempotent batch endpoint
```

6.4 Prototype policy (pseudocode, implemented in Python in Appendix A)

```
FUNCTION ChoosePolicy(batt, charging, net, motion, hot, emergency):
  IF emergency:                      RETURN 3 s,  PRIORITY_UPLOAD
  IF hot:                            RETURN 90 s, DEFER_BULK
  IF batt <= 5 AND NOT charging:     RETURN 180 s, DEFER_BULK
  IF net == OFFLINE:                 RETURN (20 s if moving else 90 s), ENCRYPT_AND_QUEUE
  IF charging AND net == WIFI:       RETURN (10 s if moving else 30 s), SYNC_PENDING_BATCH
  IF batt <= 15:                     RETURN (45 s if moving else 90 s), DEFER_BULK
  RETURN (15 s if moving else 45 s), BATCH_UPLOAD
# every record: AES-GCM encrypt before queue; upload batches of 20 over TLS;
```

```
# mark a record synced only after the server acknowledgement
```

6.5 Six-condition scenario table (MEASURED: prototype output, 1 hour per condition)

Samples = 3,600 s ÷ interval. Radio uploads assume batches of 20 records, one deferred-window upload per hour for DEFER_BULK, one upload per sample in emergency, and none while offline. The always-on baseline samples every 10 s and makes one radio attempt per sample.

Condition	Interval (s)	Action	Samples / hour	Uploads / hour	Baseline samples	Baseline radio attempts
Charging + Wi-Fi + moving	10	SYNC_PENDING_BATCH	360	18	360	360
80% + LTE + moving	15	BATCH_UPLOAD	240	12	360	360
20% + stationary + LTE	45	BATCH_UPLOAD	80	4	360	360
Low-power mode (15%) + Wi-Fi + stationary	90	DEFER_BULK	40	1	360	360
No connectivity + moving	20	ENCRYPT_AND_QUEUE	180	0	360	360
Device hot + cellular	90	DEFER_BULK	40	1	360	360
Emergency (any state)	3	PRIORITY_UPLOAD	1200	1200	360	360

Note: at 20% battery the prototype is still in the NORMAL band (the LOW threshold is 15%), so only the stationary interval changes (45 s). The intervals are design parameters, not optimum values. Energy in mAh is not stated, because no power model or device measurement exists yet. [DEVICE TEST – Battery Historian figure per condition]

6.6 Validation against an always-on baseline (MEASURED simulation)

Method: a 10-hour shift made of 15 random segments of 30, 45 or 60 minutes (seed 192425114), each with random network, motion, hot and emergency state; battery assumed to fall linearly from 100% to 15% over the shift. Offline time in this run: 270 minutes. Baseline: 10 s sampling and a radio attempt every sample, including failed retries when offline.

Policy	Samples	Radio uploads / attempts
Always-on baseline	3600	3600
Adaptive policy	945	33

The adaptive policy produced 74% fewer samples and 99.1% fewer radio wake-ups. This is a count of events, not an energy figure, and fewer samples also means lower location resolution, which is the trade-off accepted for battery life. Independent check: 60 minutes stationary at 90 s intervals should give 3,600 ÷ 90 = 40 samples, and the table above shows 40.

6.7 Scalability and load (MEASURED)

Policy evaluations on randomly generated states; time is the median of 5 runs in Python on the sandbox CPU.

Records evaluated	Median time (ms)	Mean sampling interval (s)	Emergency decisions
100	0.011	45.7	3
1000	0.096	44.6	18
10000	1.31	47.9	200

Devices in the fleet	Samples in a 10-h shift	Adaptive radio uploads	Baseline radio attempts
1	945	33	3600
5	4725	165	18000
10	9450	330	36000
20	18900	660	72000

The policy costs a negligible amount of CPU (about 1.3 ms for 10,000 evaluations), so decision logic is not the bottleneck; the radio and sensors are. Fleet figures are the single-device simulation multiplied by the device count and do not model server load. Batching cuts connection set-up cost, but continuous sensing still uses energy for the sensor, CPU, storage writes and data volume. [DEVICE TEST – wakeups and battery drop per load]

6.8 Changed requirement: 5,000 devices and emergency updates within seconds

The goal changes from energy per shift to delivery within a few seconds. The deferred-upload rules at low battery now conflict with the goal. I would keep the battery-aware mode for normal operation and add an explicit, time-limited emergency mode with a foreground tracking service, a high-priority push or persistent channel, immediate authenticated upload and fleet-side coordination, with a cellular fallback. Costs: more energy, heat, data charges and privacy exposure from frequent location sharing, so emergency mode must be switched on deliberately, logged, visible to the user and organisation, and time-limited. The server also has to handle up to 5,000 devices at 3-second intervals, about 1,700 updates per second, which has to be load-tested.

6.9 Technical reflection (under 150 words)

No single power technique suits every app. Batching cuts radio wake-ups for offline or low-priority tracking, but fails when an emergency needs a position within seconds. DVFS saves CPU energy under light load, yet a sustained graphics workload may need full speed to avoid dropped frames. Doze saves most when the phone is idle, but continuous tracking may need a justified foreground service and short wakelocks. My simulation showed the adaptive policy cutting radio wake-ups by about 99% against an always-on loop, but at the price of fewer samples. The right policy depends on battery, motion, network, temperature, queue and business priority, and its thresholds must be set from measured wakeups, CPU time, radio-active time and delivery delay.

Final submission evidence checklist

- Device or emulator model and Android build number.
- Part II Task 1: adb outputs, lifecycle log, screenshots.
- Part II Task 2: both implementations and profiler/Perfetto traces (replace sandbox timings with device timings).
- Part II Task 3: ART compile outputs, GC log, heap figures, failure-injection evidence.
- Part II Task 4: real manifest, audit output, exported-component test output and fix.
- Part II Task 5: batterystats / Battery Historian for each technique and each conflict test.
- Part II Task 6: prototype source, six-condition output, load numbers and, if available, device energy readings.
- Every cell marked [DEVICE TEST] replaced with a real observation.

Appendix A – Programs used for MEASURED results

sim2.py (Python 3) contains: the three thread benchmarks (Part II Task 2), the manifest auditor and the before/after manifests (Task 4), the ChoosePolicy function, the one-hour scenario table, the 10-hour shift simulation, the load test and the fleet scaling (Task 6). All use seed 192425114. Re-run it to reproduce the numbers; timings will differ slightly on another machine.

OVERALL CONCLUSION

Mobile operating systems differ in kernel, runtime, lifecycle, security and power policy. Android layers a Linux kernel, HAL, native libraries, ART and framework; iOS uses XNU with system frameworks and the Objective-C/Swift runtime; Windows 8 uses the NT kernel and WinRT. These differences change how processes start, how background messages arrive, how memory is managed and how IPC works.

A secure app needs least-privilege permissions, sandboxed and encrypted storage, trusted signing and authenticated transport. Battery life needs coordinated choices across CPU frequency, OS idle control, wakelocks, sensor batching and network scheduling. For RuralCare, native smartphone apps give the strongest secure offline storage and sync, while legacy feature phones need a restricted workflow.

The tracking prototype chooses its sampling and upload behaviour from battery, network, motion, temperature and emergency state, encrypts before queueing and uploads with idempotent, acknowledged batches. The sandbox experiments support the main design points (thread overhead, exported-component exposure, radio wake-up reduction), but the claims about Android behaviour must still be confirmed with device measurements, as the checklist requires.

REFERENCES

- ITA03 – Mobile Computing, Assignment 4A and 4B, supplied assignment brief.
- Android Developers documentation: processes and app lifecycle, Android Runtime (ART), application sandbox, permissions, Keystore, WorkManager, Doze and App Standby.
- Apple Developer Documentation: app lifecycle, background execution, Keychain Services, Data Protection, APNs.
- Microsoft documentation: Windows Runtime, app lifecycle, AppContainer, Windows Push Notification Services.
- Android platform tools documentation: adb, dumpsys, logcat, package compilation, batterystats.
- Perfetto documentation: system tracing and performance analysis.